



DL LAB 08

CONVOLUTIONAL NEURAL NETWORK

```
In [ ]: from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
        from tensorflow.keras.models import Sequential
        import tensorflow as tf
        from tensorflow.keras.datasets import mnist
        from tensorflow.keras.utils import to_categorical
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense, Flatten
        import numpy as np

        # Load dataset
        (X_train, y_train), (X_test, y_test) = mnist.load_data()
        # Normalize
        X_train = X_train / 255.0
        X_test = X_test / 255.0
        # reshape for CNN
        X_train_cnn = X_train.reshape(-1,28,28,1)
        X_test_cnn = X_test.reshape(-1,28,28,1)

        model = Sequential([

            Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
            MaxPooling2D((2,2)),

            Conv2D(64, (3,3), activation='relu'),
            MaxPooling2D((2,2)),

            Flatten(),

            Dense(128, activation='relu'),
            Dense(10, activation='softmax')

        ])

        model.compile(optimizer='adam',
                      loss='sparse_categorical_crossentropy',
                      metrics=['accuracy'])

        model.summary()

        model.fit(X_train_cnn, y_train,
                  epochs=10,
                  batch_size=64,
                  validation_split=0.1)

        test_loss, test_acc = model.evaluate(X_test_cnn, y_test)

        print("Test accuracy:", test_acc)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10
844/844  **46s** 52ms/step - accuracy: 0.9498 - loss: 0.1699 - val_accuracy: 0.9852 - val_loss: 0.0517
Epoch 2/10
844/844  **82s** 53ms/step - accuracy: 0.9840 - loss: 0.0522 - val_accuracy: 0.9890 - val_loss: 0.0406
Epoch 3/10
844/844  **94s** 67ms/step - accuracy: 0.9890 - loss: 0.0336 - val_accuracy: 0.9885 - val_loss: 0.0383
Epoch 4/10
844/844  **45s** 54ms/step - accuracy: 0.9918 - loss: 0.0259 - val_accuracy: 0.9897 - val_loss: 0.0343
Epoch 5/10
844/844  **44s** 52ms/step - accuracy: 0.9938 - loss: 0.0194 - val_accuracy: 0.9913 - val_loss: 0.0320
Epoch 6/10
844/844  **44s** 53ms/step - accuracy: 0.9951 - loss: 0.0152 - val_accuracy: 0.9922 - val_loss: 0.0324
Epoch 7/10
844/844  **45s** 53ms/step - accuracy: 0.9962 - loss: 0.0114 - val_accuracy: 0.9905 - val_loss: 0.0360
Epoch 8/10
844/844  **80s** 52ms/step - accuracy: 0.9968 - loss: 0.0095 - val_accuracy: 0.9905 - val_loss: 0.0378
Epoch 9/10
844/844  **84s** 55ms/step - accuracy: 0.9969 - loss: 0.0092 - val_accuracy: 0.9928 - val_loss: 0.0319
Epoch 10/10
844/844  **49s** 59ms/step - accuracy: 0.9977 - loss: 0.0070 - val_accuracy: 0.9917 - val_loss: 0.0390
313/313  **3s** 11ms/step - accuracy: 0.9901 - loss: 0.0357
Test accuracy: 0.9901000261306763

accuracy and loss graph , confusion matrix , 1 convolution operation and pooling operation deleting and get the output(2nd network) { 1- convolution 1- pooling 2- dense}, delete 128 dense layer(3rd network) {1- convolution 1- pooling 1- dense}

```
In [ ]: # add accuracy and loss graph, confusion matrix, remove second convolution operation
```

```
In [ ]: # ACCURACY GRAPH
import matplotlib.pyplot as plt
history = model.fit(X_train_cnn, y_train,
                    epochs=10,
                    batch_size=64,
                    validation_split=0.1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train Accuracy', 'Validation Accuracy'])
```

```
plt.show()
```

Epoch 1/10

844/844  **44s** 52ms/step - accuracy: 0.9983 - loss: 0.0059 - val_accuracy: 0.9923 - val_loss: 0.0392

Epoch 2/10

844/844  **45s** 53ms/step - accuracy: 0.9980 - loss: 0.0056 - val_accuracy: 0.9925 - val_loss: 0.0374

Epoch 3/10

844/844  **81s** 52ms/step - accuracy: 0.9983 - loss: 0.0058 - val_accuracy: 0.9922 - val_loss: 0.0359

Epoch 4/10

844/844  **44s** 52ms/step - accuracy: 0.9985 - loss: 0.0039 - val_accuracy: 0.9927 - val_loss: 0.0421

Epoch 5/10

844/844  **81s** 52ms/step - accuracy: 0.9986 - loss: 0.0039 - val_accuracy: 0.9930 - val_loss: 0.0435

Epoch 6/10

844/844  **83s** 53ms/step - accuracy: 0.9991 - loss: 0.0030 - val_accuracy: 0.9922 - val_loss: 0.0552

Epoch 7/10

844/844  **44s** 52ms/step - accuracy: 0.9984 - loss: 0.0047 - val_accuracy: 0.9932 - val_loss: 0.0465

Epoch 8/10

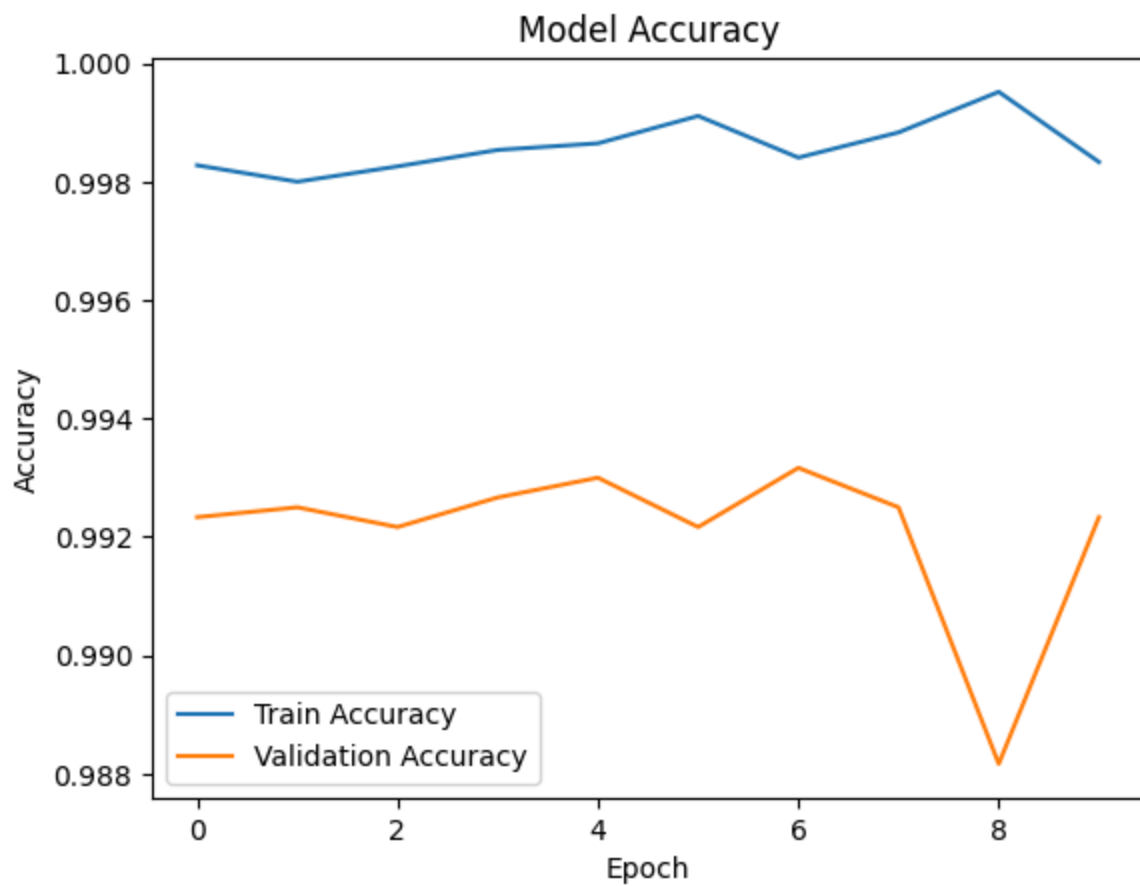
844/844  **82s** 52ms/step - accuracy: 0.9988 - loss: 0.0038 - val_accuracy: 0.9925 - val_loss: 0.0433

Epoch 9/10

844/844  **84s** 54ms/step - accuracy: 0.9995 - loss: 0.0014 - val_accuracy: 0.9882 - val_loss: 0.0813

Epoch 10/10

844/844  **44s** 52ms/step - accuracy: 0.9983 - loss: 0.0049 - val_accuracy: 0.9923 - val_loss: 0.0436

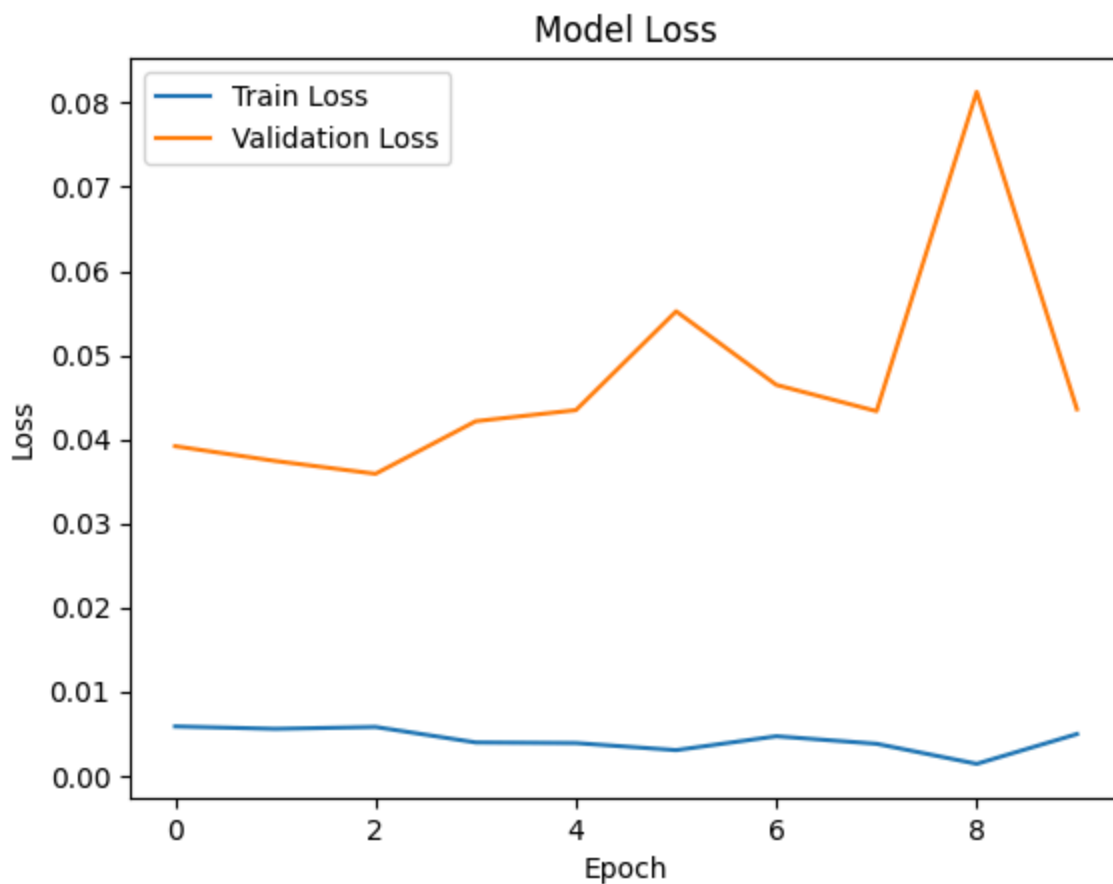


```
In [ ]: # LOSS GRAPH
import matplotlib.pyplot as plt

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train Loss', 'Validation Loss'])

plt.show()
```



```
In [ ]: # CONFUSION MATRIX
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

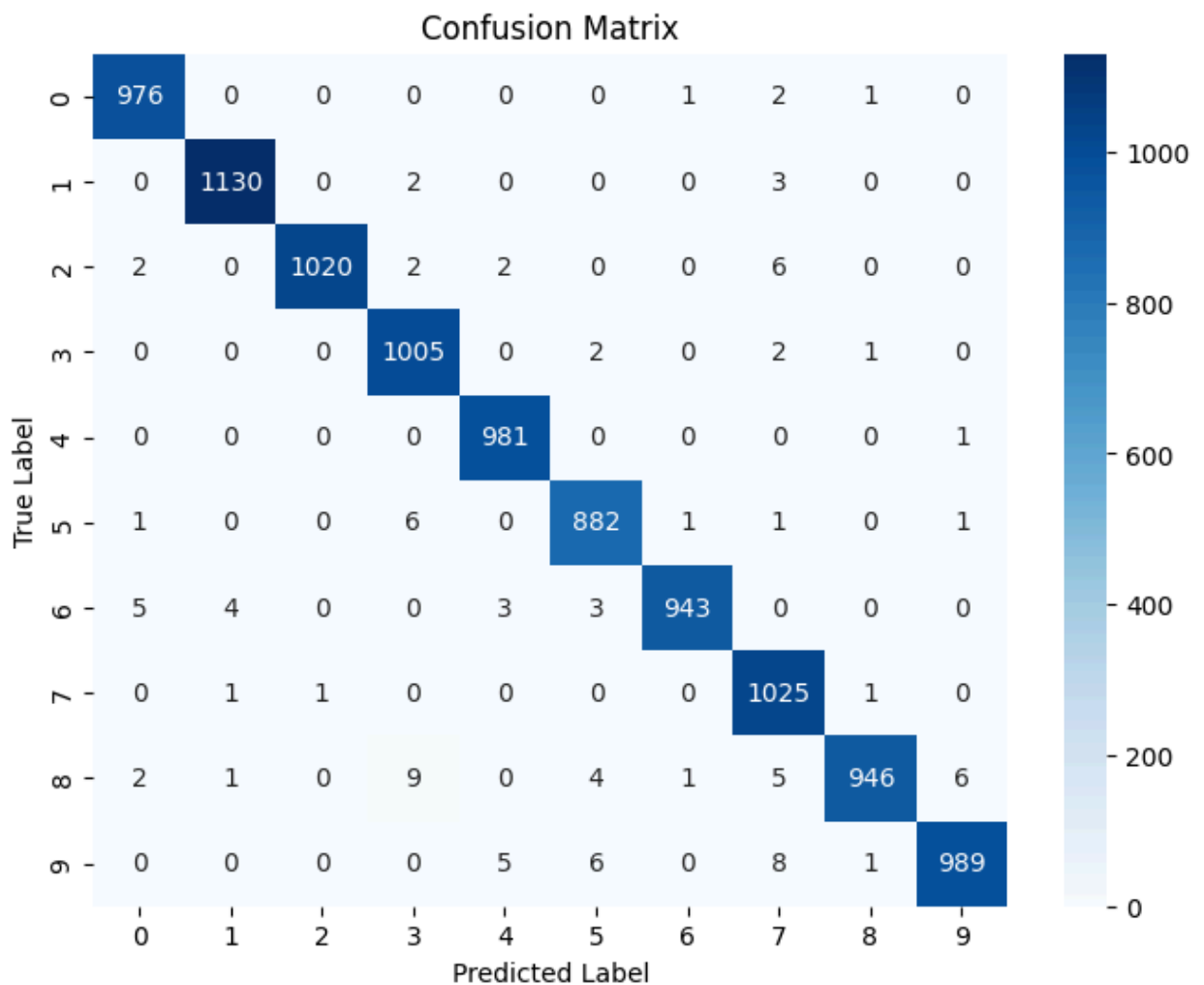
# Predict classes
y_pred = model.predict(X_test_cnn)
y_pred_classes = np.argmax(y_pred, axis=1)

# Create confusion matrix
cm = confusion_matrix(y_test, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")

plt.show()
```



```
In [ ]: # removing second convolution operation from cnn
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.models import Sequential
import tensorflow as tf
from tensorflow.keras.datasets import mnist
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape for CNN
X_train_cnn = X_train.reshape(-1, 28, 28, 1)
X_test_cnn = X_test.reshape(-1, 28, 28, 1)

# CNN Model (Single Convolution Layer)
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
```

```

MaxPooling2D((2,2)),

Flatten(),

Dense(128, activation='relu'),
Dense(10, activation='softmax')

])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train_cnn, y_train,
          epochs=10,
          batch_size=64,
          validation_split=0.1)

test_loss, test_acc = model.evaluate(X_test_cnn, y_test)

print("Test accuracy:", test_acc)

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 32)	0
flatten_1 (Flatten)	(None, 5408)	0
dense_2 (Dense)	(None, 128)	692,352
dense_3 (Dense)	(None, 10)	1,290

Total params: 693,962 (2.65 MB)

Trainable params: 693,962 (2.65 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10
844/844  **38s** 44ms/step - accuracy: 0.9414 - loss: 0.1983 - val_accuracy: 0.9777 - val_loss: 0.0761
Epoch 2/10
844/844  **33s** 34ms/step - accuracy: 0.9805 - loss: 0.0651 - val_accuracy: 0.9847 - val_loss: 0.0526
Epoch 3/10
844/844  **41s** 35ms/step - accuracy: 0.9864 - loss: 0.0442 - val_accuracy: 0.9868 - val_loss: 0.0478
Epoch 4/10
844/844  **32s** 38ms/step - accuracy: 0.9903 - loss: 0.0309 - val_accuracy: 0.9885 - val_loss: 0.0440
Epoch 5/10
844/844  **31s** 37ms/step - accuracy: 0.9929 - loss: 0.0220 - val_accuracy: 0.9870 - val_loss: 0.0515
Epoch 6/10
844/844  **38s** 34ms/step - accuracy: 0.9945 - loss: 0.0164 - val_accuracy: 0.9865 - val_loss: 0.0520
Epoch 7/10
844/844  **29s** 34ms/step - accuracy: 0.9964 - loss: 0.0113 - val_accuracy: 0.9838 - val_loss: 0.0536
Epoch 8/10
844/844  **29s** 35ms/step - accuracy: 0.9972 - loss: 0.0095 - val_accuracy: 0.9858 - val_loss: 0.0598
Epoch 9/10
844/844  **41s** 35ms/step - accuracy: 0.9978 - loss: 0.0072 - val_accuracy: 0.9893 - val_loss: 0.0536
Epoch 10/10
844/844  **29s** 34ms/step - accuracy: 0.9977 - loss: 0.0067 - val_accuracy: 0.9878 - val_loss: 0.0584
313/313  **2s** 6ms/step - accuracy: 0.9870 - loss: 0.0449
Test accuracy: 0.9869999885559082

```
In [ ]: # removing dense layer of 128 neurons from cnn
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.models import Sequential
import tensorflow as tf
from tensorflow.keras.datasets import mnist
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape for CNN
X_train_cnn = X_train.reshape(-1,28,28,1)
X_test_cnn = X_test.reshape(-1,28,28,1)

# CNN Model (Dense 128 layer removed)
model = Sequential([
```

```

Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
MaxPooling2D((2,2)),

Conv2D(64, (3,3), activation='relu'),
MaxPooling2D((2,2)),

Flatten(),

Dense(10, activation='softmax')

])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train_cnn, y_train,
          epochs=10,
          batch_size=64,
          validation_split=0.1)

test_loss, test_acc = model.evaluate(X_test_cnn, y_test)

print("Test accuracy:", test_acc)

```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_3 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_4 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_4 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_2 (Flatten)	(None, 1600)	0
dense_4 (Dense)	(None, 10)	16,010

Total params: 34,826 (136.04 KB)

Trainable params: 34,826 (136.04 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10
844/844  **43s** 50ms/step - accuracy: 0.9402 - loss: 0.2054 - val_accuracy: 0.9815 - val_loss: 0.0636
Epoch 2/10
844/844  **43s** 51ms/step - accuracy: 0.9797 - loss: 0.0647 - val_accuracy: 0.9862 - val_loss: 0.0477
Epoch 3/10
844/844  **42s** 50ms/step - accuracy: 0.9848 - loss: 0.0476 - val_accuracy: 0.9877 - val_loss: 0.0411
Epoch 4/10
844/844  **42s** 49ms/step - accuracy: 0.9884 - loss: 0.0375 - val_accuracy: 0.9898 - val_loss: 0.0366
Epoch 5/10
844/844  **42s** 50ms/step - accuracy: 0.9901 - loss: 0.0321 - val_accuracy: 0.9877 - val_loss: 0.0380
Epoch 6/10
844/844  **42s** 50ms/step - accuracy: 0.9915 - loss: 0.0267 - val_accuracy: 0.9892 - val_loss: 0.0394
Epoch 7/10
844/844  **42s** 49ms/step - accuracy: 0.9930 - loss: 0.0213 - val_accuracy: 0.9903 - val_loss: 0.0350
Epoch 8/10
844/844  **82s** 49ms/step - accuracy: 0.9947 - loss: 0.0172 - val_accuracy: 0.9883 - val_loss: 0.0432
Epoch 9/10
844/844  **42s** 49ms/step - accuracy: 0.9949 - loss: 0.0153 - val_accuracy: 0.9860 - val_loss: 0.0512
Epoch 10/10
844/844  **42s** 50ms/step - accuracy: 0.9959 - loss: 0.0125 - val_accuracy: 0.9913 - val_loss: 0.0370
313/313  **3s** 8ms/step - accuracy: 0.9904 - loss: 0.0309
Test accuracy: 0.9904000163078308

```
In [ ]: import tensorflow as tf
        from tensorflow.keras.preprocessing.image import ImageDataGenerator
        from tensorflow.keras import layers, models

        # Image size & batch
        IMG_SIZE = 150
        BATCH_SIZE = 32

        # Data generators (auto loading images from folders)
        train_datagen = ImageDataGenerator(
            rescale=1./255,
            validation_split=0.2
        )

        train_data = train_datagen.flow_from_directory(
            'dataset/train',
            target_size=(IMG_SIZE, IMG_SIZE),
            batch_size=BATCH_SIZE,
            class_mode='binary',
            subset='training'
        )
```

```

val_data = train_datagen.flow_from_directory(
    'dataset/train',
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='validation'
)

test_data = ImageDataGenerator(rescale=1./255).flow_from_directory(
    'dataset/test',
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='binary'
)

# CNN Model
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(150,150,3)),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(128, (3,3), activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Flatten(),

    layers.Dense(128, activation='relu'),
    layers.Dense(1, activation='sigmoid') # binary classification
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()

# Train
history = model.fit(
    train_data,
    epochs=10,
    validation_data=val_data
)

# Evaluate
test_loss, test_acc = model.evaluate(test_data)
print("Test Accuracy:", test_acc)

```